

OCT Anomaly Detection

Edison Ang (1006910)¹ and Stavva Sharma (1006995)²

¹Information Systems Technology and Design (ISTD) Pillar

Singapore University of Technology and Design

1 Introduction

Optical Coherence Tomography (OCT) is a non-invasive imaging modality essential for the diagnosis and management of various retinal diseases. As the volume of OCT scans generated in clinical settings continues to grow, there is an increasing demand for automated screening systems that can reliably distinguish between normal retinal tissue and various pathological anomalies.

In this project, we propose a hybrid deep learning framework for retinal anomaly detection. Our approach utilizes a U-Net based Autoencoder architecture for high-dimensional feature extraction and image reconstruction. By training the model purely on normal retinal images, the network learns to reconstruct standard anatomical structures with high fidelity. During the inference phase, the model’s inability to accurately reconstruct pathological features—such as fluid pockets or drusen—manifests as a higher reconstruction error, measured through Structural Similarity Index (SSIM) and L1 loss.

To enhance classification performance, we extract the latent embeddings from the bottleneck of the Autoencoder and feed them into a dynamically tuned Multi-Layer Perceptron (MLP). This allows the system to not only rely on pixel-wise reconstruction errors but also on the semantic feature representations of the retinal morphology. We evaluate our model on the Kermany 2018 dataset, specifically focusing on its efficacy in a binary "Normal vs. Anomaly" screening task, and compare its performance against established baseline architectures using metrics such as F1-score, Area Under the ROC Curve (AUC), and specificity.

2 Dataset

For this project, we utilize the "Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for Classification" dataset, originally introduced by Kermany et al.

The dataset can be accessed here: [Mendeley Data: OCT2017 Dataset](#).

The dataset consists of 84,495 high-resolution B-scan OCT images acquired from various clinical sites. The images are categorized into four distinct classes:

- **Normal:** Retinal scans with no visible pathology or abnormalities.
- **Choroidal Neovascularization (CNV):** Evidence of new blood vessel growth beneath the retina.
- **Diabetic Macular Edema (DME):** Presence of intraretinal fluid or cysts.
- **Drusen:** Small yellow deposits under the retina, early signs of macular degeneration.

While the original dataset is designed for a four-class classification problem, our project focuses on a binary anomaly detection task. We define the *Normal* class as the "negative" (healthy) samples and aggregate the *CNV*, *DME*, and *Drusen* classes into a single *Anomaly* category representing the "positive" (pathological) samples.

Also, The original dataset consist of the following distribution summarized in the following table.

Table 1: Distribution of original OCT Dataset across Training and Testing Splits

Split	Normal	DRUSEN	DME	CNV	Total
Train	51,140	8,616	11,348	37,205	108,309
Test	250	250	250	250	1,000
Total	51,390	8,866	11,598	37,455	109,309

To align the dataset with an unsupervised anomaly detection framework, we restricted the training set exclusively to Normal (healthy) samples. As summarized in Table 2, we utilized 40,000 Normal scans to train the autoencoder backbone, forcing the model to learn a compact representation of healthy retinal anatomy.

For the secondary Phase 2 classification head (CNN or Latent MLP), we utilized the 10,450 samples in our test set. This set was subjected to a 50 percent stratified split, where one half was used to train the classification head on the extracted reconstruction errors or latent features, and the remaining half served as the final unseen evaluation set. Under this distribution, the testing environment maintains an anomaly prevalence of approximately 4.3 percent, providing a rigorous benchmark that reflects the imbalance typically found in clinical screening scenarios.

Table 2: Revised OCT Dataset Distribution with Restricted Training Normal Class

Split	Normal	DRUSEN	DME	CNV	Total
Train	40,000	0	0	0	40,000
Test	5,000	75	75	75	5,225
Validation	5,000	75	75	75	5,225
Total	50,000	150	150	150	50,450

3 State of the Arts Model (PatchCore)

PatchCore is selected as the state-of-the-art reference model due to its strong empirical performance in anomaly detection, particularly for detecting subtle and localized abnormalities.

Backbone: Pre-trained ResNet18 with features extracted from intermediate layers (layer2 and layer3).

Feature representation: Multi-scale convolutional feature maps aggregated and downsampled via average pooling.

Memory bank: Constructed using patch-level features from normal training samples, with 1% subsampling for efficiency.

Dimensionality reduction: PCA applied to compress feature representations before nearest-neighbor search.

Scoring mechanism: k-nearest neighbor (k=1) distance used to compute anomaly scores for each patch.

Input: OCT image repeated to three channels and resized to 224×224 .

Output: Patch-level anomaly map and scalar anomaly score obtained via aggregation of patch distances.

Unlike reconstruction-based models, this approach directly compares feature representations to a memory bank of normal patterns. This enables detection of fine-grained deviations without relying on reconstruction quality.

By operating at the patch level, the model is particularly effective for retinal OCT images, where anomalies are often small and spatially localized.

Table 3: Performance Metrics for PatchCore (SOTA Reference Model)

F1	Precision	Recall	Specificity	AUC-ROC	AUC-PR
0.400	0.370	0.433	0.967	0.884	0.377

4 Architectures Explored

In this research, we developed and compared multiple architectures for OCT anomaly detection, spanning pixel-reconstruction baselines, two-stage hybrid U-Net pipelines, and feature-space anomaly models based on convolutional and transformer backbones. The models were designed to evaluate different anomaly detection principles: direct reconstruction error, spatial analysis of residual maps, latent-space classification, and feature-distribution deviation.

4.1 Reconstruction-Based Baseline

4.1.1 Vanilla Convolutional Autoencoder (CAE)

Backbone: Custom convolutional encoder-decoder with four convolutional downsampling layers and four transpose-convolution upsampling layers.

Head classifier: None. Anomaly detection is based on reconstruction error alone.

Loss function: Mean Squared Error (MSE).

Input: Grayscale OCT image resized to 224×224 .

Output: Reconstructed OCT image of the same size.

This model serves as the simplest baseline. It is trained only on normal OCT images so that

it learns to reconstruct healthy retinal structure. During inference, abnormal samples are expected to reconstruct poorly, leading to higher reconstruction error than normal scans. A threshold is then applied to the reconstruction error to classify each sample as normal or anomalous.

Table 4: Performance Metrics for Vanilla Convolutional Autoencoder

F1	Precision	Recall	Specificity	AUC-ROC	AUC-PR
0.274	0.185	0.524	0.896	0.848	0.172

4.2 U-Net Hybrid Architectures

4.2.1 U-Net + CNN

Backbone: U-Net with a ResNet34 encoder pre-trained on ImageNet.

Head classifier: ErrorCNN operating on the reconstruction residual map.

Loss function:

- Stage 1 (U-Net): $1 - \text{SSIM}(x, \hat{x})$
- Stage 2 (CNN): Binary Cross-Entropy (BCE)

Input: Grayscale OCT image resized to 224×224 .

Output:

- Stage 1: reconstructed OCT image
- Stage 2: anomaly probability

This architecture is a two-stage pipeline. First, the U-Net is trained on normal OCT images to reconstruct healthy retinal anatomy. Then, the absolute reconstruction residual map $|x - \hat{x}|$ is generated and passed to a secondary CNN classifier. The motivation is that the residual map preserves spatial information about where the autoencoder fails, allowing the CNN to distinguish between harmless reconstruction noise and pathologically meaningful error patterns.

Table 5: Performance Metrics for U-Net + CNN

F1	Precision	Recall	Specificity	AUC-ROC	AUC-PR
0.317	0.286	0.356	0.960	0.804	0.264

4.2.2 U-Net + Latent MLP

Backbone: U-Net with a ResNet34 encoder pre-trained on ImageNet.

Head classifier: Multi-Layer Perceptron (Latent MLP) on pooled latent features.

Loss function:

- Stage 1 (U-Net): $1 - \text{SSIM}(x, \hat{x})$
- Stage 2 (Latent MLP): Binary Cross-Entropy (BCE)

Input: Grayscale OCT image resized to 224×224 .

Output:

- Stage 1: reconstructed OCT image
- Stage 2: anomaly probability

Instead of using the spatial residual map, this model extracts the deepest encoder feature map from the trained U-Net and applies adaptive average pooling to obtain a compact latent vector. This vector is then used as input to an MLP classifier. The logic is that training the U-Net only on normal scans encourages the latent space to encode healthy anatomy, making anomalous samples appear as deviations in latent feature space.

Table 6: Performance Metrics for U-Net + Latent MLP

F1	Precision	Recall	Specificity	AUC-ROC	AUC-PR
0.759	0.839	0.693	0.994	0.972	0.815

4.3 Transformer and Feature-Space Models

4.3.1 ViT Centroid Distance

Backbone: Pre-trained Vision Transformer (ViT-B/16).

Head classifier: None. Distance to the normal centroid is used as the anomaly score.

Loss function: No trainable classifier loss in the detection stage. The anomaly score is based on Euclidean distance in embedding space.

Input: OCT image repeated to three channels and resized to 224×224 .

Output: 768-dimensional feature embedding and scalar anomaly score.

In this approach, a pre-trained ViT is used purely as a feature extractor. Normal training samples are projected into the embedding space, and their mean vector is computed as the normal centroid. At test time, the Euclidean distance between a sample embedding and the normal centroid is used as the anomaly score. The underlying assumption is that anomalous images lie farther from the normal distribution in transformer feature space.

4.3.2 ViT Feature Autoencoder

Backbone: Pre-trained Vision Transformer (ViT-B/16).

Head classifier: Fully connected feature autoencoder operating on 768-dimensional ViT embeddings.

Loss function: $1 - \text{SSIM}$ applied after reshaping embeddings into a 24×32 feature map.

Input: OCT image repeated to three channels and resized to 224×224 .

Output: Reconstructed ViT embedding and scalar reconstruction-based anomaly score.

Rather than reconstructing raw pixels, this model reconstructs the transformer embedding itself. The intuition is that the pre-trained ViT already extracts high-level semantic structure, and abnormalities may be easier to detect in this compact feature space than in the original image domain. Samples with poorly reconstructed embeddings are considered more likely to be anomalous.

Table 7: Performance Metrics for ViT Feature Autoencoder

F1	Precision	Recall	Specificity	AUC-ROC	AUC-PR
0.350	0.374	0.329	0.975	0.865	0.364

4.3.3 PatchCore Feature Autoencoder

Backbone: Pre-trained ResNet18 using intermediate PatchCore-style features from Layer 2 and Layer 3.

Head classifier: Convolutional feature autoencoder operating on pooled PatchCore feature maps.

Loss function: $1 - \text{SSIM}$ on reconstructed feature maps.

Input: OCT image repeated to three channels and resized to 224×224 .

Output: Reconstructed feature map and scalar reconstruction-based anomaly score.

This model first extracts intermediate convolutional features instead of reconstructing the raw image. Layer 2 and Layer 3 feature maps are aligned and pooled to form a PatchCore-style representation, which is then reconstructed by a lightweight convolutional autoencoder. The motivation is that anomalies may be more distinguishable in deep convolutional feature space than at the pixel level.

Table 8: Performance Metrics for ViT Feature Autoencoder

F1	Precision	Recall	Specificity	AUC-ROC	AUC-PR
0.468	0.441	0.498	0.972	0.903	0.462

4.4 Tuned Final Model

4.4.1 Best Tuned U-Net + Latent MLP

Backbone: U-Net with ResNet50 encoder.

Head classifier: Latent MLP with hidden layers [512, 256, 128, 64] and dropout rate 0.1.

Loss function:

- Autoencoder stage: weighted SSIM + L1 reconstruction loss during tuning
- MLP stage: Binary Cross-Entropy (BCE)

Input: Grayscale OCT image resized to 224×224 .

Output: anomaly probability, followed by threshold-based binary prediction.

The final selected model was obtained through a phased tuning process that searched over encoder backbones, reconstruction-loss weighting, and MLP architectures. The winning configuration used a ResNet50-based U-Net encoder and a latent-feature MLP head. This final design preserves the two-stage logic of the U-Net + Latent MLP architecture while improving performance through systematic hyperparameter selection.

4.5 Summary of Architectural Logic

Overall, the tested models can be grouped into three categories. First, reconstruction-only models rely on the principle that abnormal images reconstruct less accurately than normal ones. Second, two-stage hybrid pipelines use a learned classifier on top of either residual maps or latent embeddings to improve discrimination beyond a single reconstruction score. Third, transformer and feature-space models operate on high-level semantic representations, testing whether anomalies are more separable in pretrained feature space than in the raw image domain.

5 Best Experiment and Result

5.1 Final Selected Configuration

The best-performing configuration used a U-Net autoencoder with a ResNet50 encoder backbone and a latent-space MLP classifier with hidden layers [512, 256, 128, 64] and dropout 0.1.

5.2 Loss Function and Tuned Hyperparameters

The final model was trained as a two-stage pipeline rather than with a single joint loss. In the first stage, the U-Net autoencoder was trained only on normal OCT images using a weighted reconstruction objective that combines SSIM loss and L1 loss:

$$\mathcal{L}_{AE} = \alpha (1 - \text{SSIM}(x, \hat{x})) + (1 - \alpha) \|x - \hat{x}\|_1 \quad (1)$$

where x is the input OCT image, $\hat{x}$ is the reconstructed image, and α controls the relative contribution of the SSIM and L1 terms.

The use of SSIM in the autoencoder training objective was motivated by the nature of OCT anomaly detection. The disease categories considered in this study, namely CNV, DME, and DRUSEN, are primarily characterized by structural changes in retinal layers rather than only simple pixel-intensity differences. SSIM is therefore more suitable than a purely pixel-wise loss because it measures similarity in local structure, contrast, and luminance between the

input and reconstructed image. By encouraging the model to preserve normal retinal organization during reconstruction, SSIM helps the encoder learn feature representations that are sensitive to disruptions in retinal morphology, which are clinically important for distinguishing normal and abnormal OCT scans.

In the second stage, latent vectors were extracted from the encoder bottleneck and used to train a latent-space MLP classifier. The classifier was optimized separately using binary cross-entropy:

$$\mathcal{L}_{\text{MLP}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2)$$

where y_i denotes the ground-truth binary label and $\hat{y}_i$ is the predicted anomaly probability.

For the best-performing model, the selected reconstruction weighting was $\alpha = 0.5$, giving equal weight to the SSIM-based and L1-based reconstruction terms during autoencoder training. The final tuned hyperparameters are summarized in Table 9.

Table 9: Summary of tuning choices for the final best model.

Component	Values Tried	Best Selected
Encoder backbone	ResNet18, ResNet34, ResNet50	ResNet50
Reconstruction loss weighting (α)	0.5, 0.8, 1.0	0.5
MLP hidden layers	[256, 64], [1024, 256], [512, 256, 128, 64], [1024, 512, 256]	[512, 256, 128, 64]
MLP dropout	0.1, 0.3, 0.5	0.1

5.3 Prediction Score Distribution

Figure 1 shows the distribution of anomaly scores for normal and anomalous test samples. This figure illustrates how well the final model separates the two classes in score space. A clearer separation indicates that the latent-space classifier is able to distinguish normal and abnormal retinal patterns effectively.

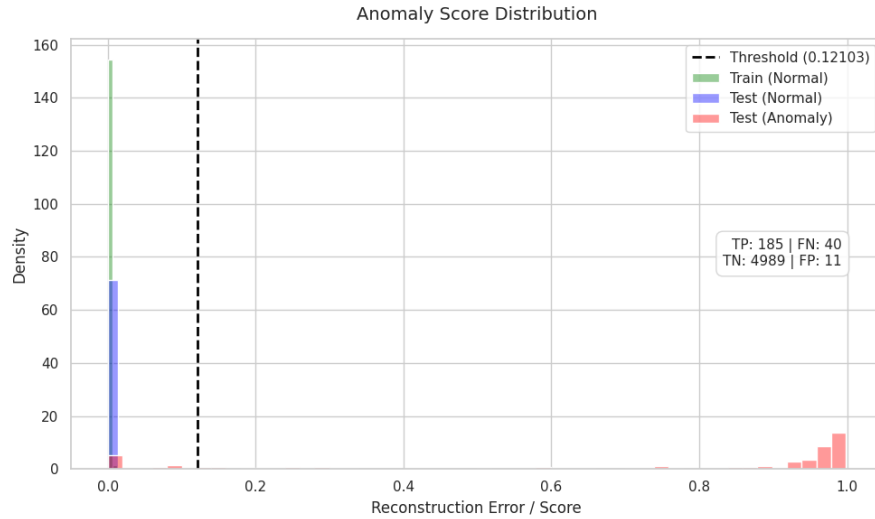


Figure 1: Distribution of predicted anomaly scores for the best model on the test set.

5.4 Confusion Matrix and Final Performance

Figure 2 presents the confusion matrix of the final selected model.

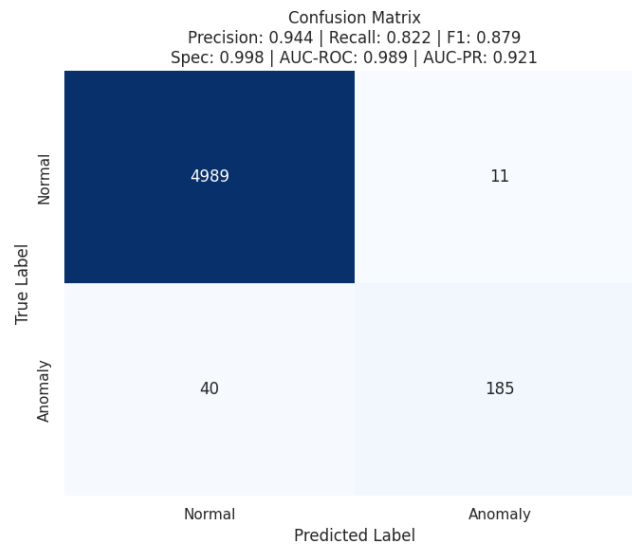


Figure 2: Confusion matrix of the final best-performing model on the test set.

The final model achieves strong overall performance, with a precision of 0.944, recall of 0.822,

F1-score of 0.879, and specificity of 0.998. The high F1-score indicates a good overall balance between sensitivity and precision, while the very high specificity shows that the model is highly effective at correctly identifying normal scans.

However, in a medical anomaly detection setting, false negatives are particularly important because they correspond to diseased cases being incorrectly classified as normal. In the current results, the model missed 40 anomalous samples while correctly identifying 185 out of 225 anomalies. This indicates that although the recall is reasonably strong, some disease cases are still not detected.

At the same time, the number of false positives is very low, with only 11 normal scans incorrectly classified as anomalous out of 5000 normal test samples. This suggests that the model is conservative and rarely over-predicts disease.

Overall, the confusion matrix shows that the model performs strongly, but reducing false negatives would be especially important for improving its suitability in medical screening applications.

Overall, these results suggest that the final model provides a good trade-off for medical anomaly detection: it is conservative when flagging disease, which is beneficial for avoiding over-diagnosis, while still achieving strong anomaly detection performance.

5.5 Prediction Breakdown by Class

Figure 3 shows the number of correctly and incorrectly classified samples for the normal class and for each disease category.

Prediction Breakdown by Class

Class	Class Type	Correct	Wrong	Total	Accuracy
NORMAL	Aggregate	4989	11	5000	0.998
ANOMALY	Aggregate	185	40	225	0.822
CNV	Disease	67	7	74	0.905
DME	Disease	59	15	74	0.797
DRUSEN	Disease	59	18	77	0.766

Figure 3: Prediction breakdown by class for the final best-performing model on the test set.

Across the anomaly classes, performance is more variable. CNV achieves the highest class-level accuracy (0.905), followed by DME (0.797) and DRUSEN (0.766). This suggests that CNV samples may be easier for the model to distinguish from normal scans, whereas DME and DRUSEN are more frequently confused with normal retinal structure.

A possible explanation is that CNV often introduces stronger or more obvious structural deviations, making it more separable in either reconstruction space or latent feature space. In contrast, DME and DRUSEN may preserve more of the overall normal retinal appearance, with abnormalities that are subtler or more localized. As a result, their latent representations may remain closer to the learned normal distribution, leading to more false negative predictions.

5.6 Examples of Misclassified Samples

To better understand the limitations of the final model, we examined representative misclassified anomaly samples from the three disease categories: DME, DRUSEN, and CNV. Figures 4, 5, and 6 present one false negative example from each class, together with the original OCT image, the reconstructed image, and the corresponding error heatmap.

Although these samples are truly anomalous, the model predicted them as normal. This is because the final decision in the best model is not made directly from the visual brightness of the reconstruction heatmap. Instead, the classification output is produced by the latent-space MLP, which operates on the pooled encoder features extracted from the U-Net bottleneck. The error heatmap is therefore used primarily as a qualitative visualization of reconstruction discrepancy rather than as the final decision rule.

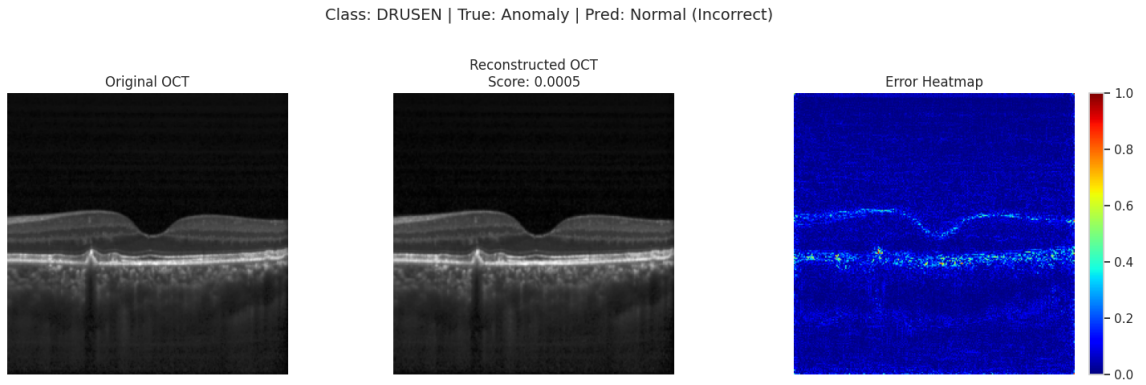


Figure 4: False negative DRUSEN example showing the original OCT image, reconstructed image, and reconstruction error heatmap.

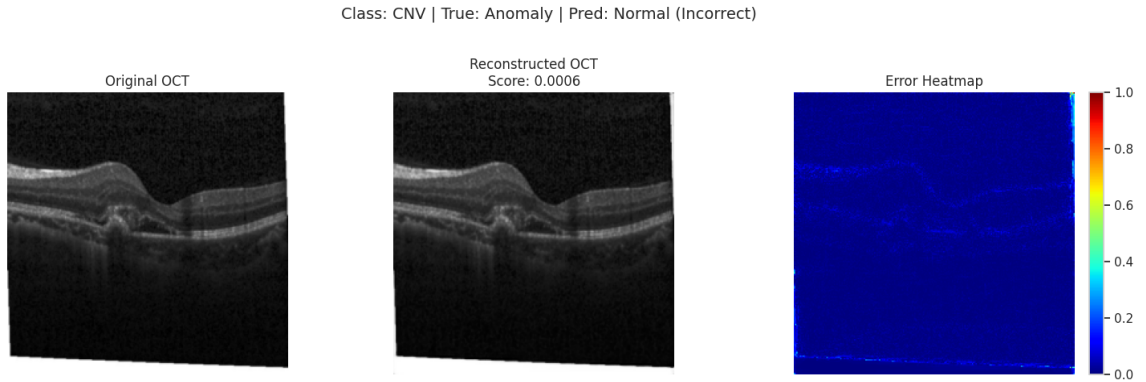


Figure 5: False negative CNV example showing the original OCT image, reconstructed image, and reconstruction error heatmap.

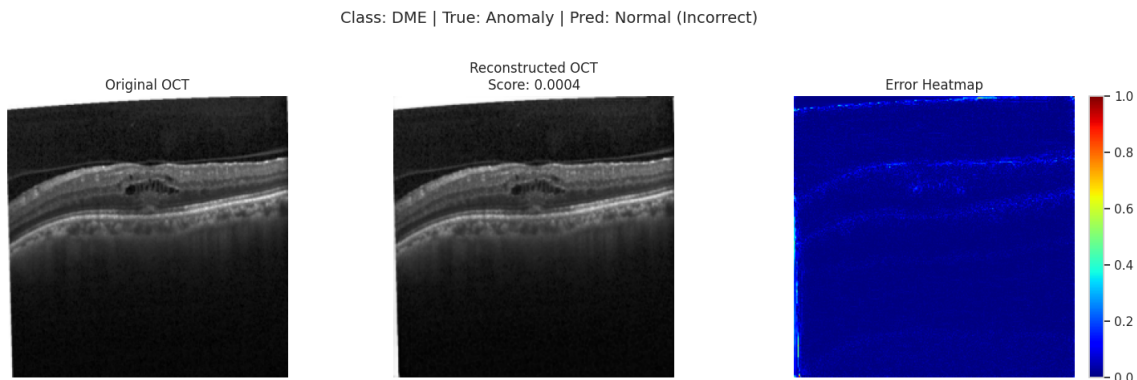


Figure 6: False negative DME example showing the original OCT image, reconstructed image, and reconstruction error heatmap.

5.7 Analysis of Failure Cases

The false negative cases show an important limitation of the final model. Even when the error heatmap looks bright, the image may still be predicted as normal. This is because the final prediction is not made from the heatmap itself. Instead, the decision is made by the latent MLP classifier, which uses the feature vector extracted from the U-Net encoder.

In addition, the heatmap is normalized separately for each image for visualization. This means that a bright region only shows where the reconstruction error is relatively larger within that sample, not that the total error is necessarily high compared with other images.

For the DRUSEN, examples, the autoencoder detected some local reconstruction differences, but the latent representation of the image still appeared similar to normal scans. As a result, the anomaly score remained below the decision threshold and the model predicted the image as normal.

For the DME and CNV examples, the failure mode appears slightly different. In these cases, the reconstruction heatmap shows only weak visible error, suggesting that the autoencoder was able to reconstruct much of the abnormal image structure successfully. As a result, the anomalies did not create a strong enough deviation in reconstruction space to clearly distinguish them from normal scans.

At the same time, the latent representation extracted by the encoder also remained insufficiently different from the learned normal feature distribution. This caused the latent MLP to assign a very low anomaly score, which remained below the decision threshold. In other words, the model not only reconstructed these abnormal scans quite well, but also represented them in latent space as being too similar to normal retinal structure.

These examples suggest that some pathological scans, especially those with subtle or structurally smooth abnormalities, may not generate strong signals in either the reconstruction error map or the latent feature space. Consequently, the final model may fail to detect them and produce false negative predictions.

6 Conclusion

In this project, we explored multiple deep learning approaches for OCT retinal anomaly detection, including reconstruction-based baselines, hybrid U-Net pipelines, and feature-space models. The results show that reconstruction-only methods are insufficient for reliable detection, particularly for subtle anomalies, as they struggle to clearly separate pathological cases from normal scans.

The final tuned model, based on a **U-Net + Latent MLP** architecture, achieved the strongest performance, with a **precision of 0.944**, **recall of 0.822**, **F1-score of 0.879**, and **specificity of 0.998**. These results demonstrate that combining reconstruction-based learning with latent-space classification is significantly more effective than relying on reconstruction error alone. In particular, latent representations capture higher-level structural patterns in retinal scans, enabling better discrimination between normal and abnormal cases.

However, failure case analysis shows that the model still struggles with subtle anomalies such as DME and DRUSEN, which are more frequently confused with normal retinal structure. These cases often remain close to the learned normal distribution, leading to false negatives.

Overall, this work shows that hybrid architectures provide a strong and effective framework for OCT anomaly detection, while also highlighting the need for improved sensitivity to subtle abnormalities.

7 Contribution

Edison Ang (1006910)

Conducted extensive model explorations across various architectures. Performed hyperparameter optimization and fine-tuning to achieve the final best-performing models.

Stavya Sharma (1006995)

Contributed to model development through experimentation, tuning, and iterative improvements on existing architectures. Supported evaluation and analysis to guide model refinement and improve overall performance.

8 Reproducibility and Training Instructions

This project repository contains the final PDF report, the documented source code, and the saved best-model artifacts required for reproducibility. The dataset itself is not included in the repository due to storage restrictions. To reproduce the experiments, the dataset must first be downloaded separately and placed into the expected directory structure.

8.1 Required Libraries

The project was developed in Python and requires the libraries listed in the repository `requirements.txt` file. The main dependencies include `torch`, `torchvision`, `numpy`, `matplotlib`, `seaborn`, `scikit-learn`,

mlflow, tqdm, torchmetrics, segmentation-models-pytorch, and Pillow.

These dependencies can be installed with:

```
pip install -r requirements.txt
```

8.2 Dataset Setup

The dataset should be arranged into separate training and testing folders following the directory structure expected by the codebase:

```
/workspace/data/train  
/workspace/data/test
```

The training set should contain only normal images for the anomaly-detection training pipeline, while the test set should contain both normal and anomalous samples from the disease categories CNV, DME, and DRUSEN.

8.3 Configuration

Before training, the dataset paths and experiment settings should be checked in the configuration files:

- `src/config.py`: main dataset paths, image size, batch size, and device settings
- `src/models/best_model_tuning/config_tune.py`: tuning phase, candidate backbones, reconstruction loss weights, MLP architectures, dropout rates, and random seed

8.4 Training the Final Model from Scratch

To retrain the final model from scratch, the main settings should first be configured in `src/models/best_model_tuning/config_tune.py`. The most important fields are:

- `CURRENT_PHASE`: controls which stage of the tuning pipeline is executed
- `BACKBONES`: candidate encoder backbones evaluated during Phase 1
- `LOSS_ALPHAS`: candidate reconstruction-loss weightings evaluated during Phase 2
- `MLP_ARCHITECTURES`: candidate latent-space MLP hidden-layer configurations evaluated during Phase 3
- `MLP_DROPOUTS`: candidate dropout values for the latent MLP
- `BEST_BACKBONE_SO_FAR`: selected best encoder backbone from Phase 1

- `BEST_AE_WEIGHTS_PATH`: selected best autoencoder checkpoint from Phase 2
- `RANDOM_SEED`: controls reproducibility of the tuning process

The final best-performing configuration obtained from this tuning process was:

- encoder backbone: `resnet50`
- reconstruction loss weighting: $\alpha = 0.5$
- latent dimension: 2048
- MLP hidden layers: [512, 256, 128, 64]
- MLP dropout: 0.1
- random seed: 42

The dataset paths, image size, batch size, and hardware configuration should also be checked in `src/config.py` before training.

Training is then performed by running:

```
python -m src.models.best_model_tuning.best_model_script
```

The script should be executed separately for Phase 1, Phase 2, and Phase 3 by updating the corresponding configuration values in `config_tune.py`.

8.5 Saved Artifacts

The final reproducible model consists of:

- the best U-Net autoencoder weights
- the best latent MLP weights
- the metadata file describing the selected configuration

These files are stored in the repository under:

```
src/inference/best_model/
```

8.6 Loading the Final Model Without Retraining

To verify that the saved model can be loaded correctly without retraining, run:

```
python -m src.inference.best_model.load_best_model ^  
--metadata src/inference/best_model/best_model_metadata.json ^  
--check-only
```

To run prediction on a single OCT image, use:

```
python -m src.inference.best_model.load_best_model ^  
--metadata src/inference/best_model/best_model_metadata.json ^  
--image path/to/image.jpeg
```